

沈阳航空航天大学

课程设计报告

课程设计名称：算法分析课程设计

课程设计题目：基于双向循环链表的通讯录的设计

学 院：人工智能学院

专 业：物联网工程

班 级：物联网2202

姓 名：陈梓欣

学 号：223428010210

指导教师：张德园

完成时间：2024 年 6 月 27 日

课程设计任务书

课程名称	算法分析课程设计		专业	物联网工程	
学生姓名	陈梓欣	班级	物联网 2202	学号	223428010210
题目名称	基于双向循环链表的通讯录的设计				
起止日期	2024 年 6 月 19 日起 至 2024 年 6 月 30 日止				

课设内容和要求：

一、课程设计内容

利用双向循环链表作为存储结构设计并实现一个通讯录程序。要求：

- (1) 通讯录包括姓名，手机号，QQ 号和邮箱即可；
- (2) 可以实现信息的添加、插入、删除、查询和统计等功能。

二、课程设计要求

1. 独立完成课程设计任务；
2. 通过老师当场验收；
3. 交出完整的课程设计

参考资料：

- [1] 严蔚敏等. 数据结构 (C 语言版). 北京: 清华大学出版社, 1997
- [2] 李春荣. 数据结构教程 (第 3 版). 北京: 清华大学出版社, 2009
- [3] 王敬华等. C 语言程序设计教程 (第二版). 北京: 清华大学出版社, 2005

教研室审核意见：同意 (☒) 不同意 (☐) 教研室主任签字：

指导教师 (签名)

2024 年 6 月 15 日

学生签名 (签名)

陈梓欣

2024 年 6 月 15 日

课程设计总结

在本次课程设计中,我设计并实现了一个通讯录管理系统,具有添加联系人、插入联系人、删除联系人、查询联系人和统计联系人数量等功能。以下是对这个通讯录管理系统的总结:

本次的通讯录管理系统实现了基本的功能,并且代码结构相对清晰,易于理解和维护。通过使用函数、数据结构和正则表达式等技术,使得程序具有一定的可扩展性和复用性。

通过使用双链表的数据结构来设计通讯录管理系统,使用 Node 结构体作为通讯录节点,包含了联系人的姓名、手机号、QQ 号和邮箱等信息,以及前指针和后指针。通过将联系人节点连接成双向循环链表的形式,实现了通讯录中联系人的管理和遍历。

通讯录管理系统采用了面向对象的设计思想,将通讯录的操作封装为不同的函数,设计了新建、删除、插入、查询和统计联系人与查看联系人具体信息的函数,并对函数做了详细的注释。在联系人手机号和邮箱的输入中增加了数据验证功能,使用了正则表达式对手机号和邮箱进行格式验证,对错误的信息进行相应的提示,确保用户输入的手机号和邮箱符合规定的格式要求。

设计了一个简单的可交互的 UI 界面,以类似软件的布局形式展示通讯录管理系统的功能操作供用户选择,提升用户的使用体验。

最后,通过本次的算法导论课程设计,我对于链表的存储结构、常用功能如添加、插入、删除和查询等功能有了更深刻的理解。同时让我明白了数据结构在生产实际中的应用,充分体验到数据结构在程序设计中的重要性。

目 录

1 引言	1
2 课程设计分析	2
2.1 问题分析	2
2.1.1 以双链表的数据结构存储	2
2.1.2 实现通讯录相关的操作	2
2.1.3 对输入的数据进行校验	2
2.1.4 设计友好的用户交互界面	2
2.2 需求分析	2
2.2.1 存储结构分析	2
2.2.2 功能分析	3
2.2.3 用户界面设计	3
2.3 算法设计	3
2.3.1 双链表数据结构	3
2.3.2 功能实现算法	3
3 程序设计	4
3.1 程序框架	4
3.2 模块设计	4
3.2.1 链表节点结构体定义	4
3.2.2 数据验证	5
3.2.3 新建联系人	6
3.2.4 插入联系人	8
3.2.5 删除联系人	10
3.2.6 查询联系人	12
3.2.7 统计联系人	14

4 系统测试	16
4.1 测试环境	16
4.2 测试数据	16
4.3 本地数据库	17
4.4 测试过程演示	18
4.4.1 通讯录管理系统页面展示:	18
4.4.2 新建联系人	20
4.4.3 错误信号验证	21
4.4.4 插入联系人	24
4.4.5 删除联系人	25
4.4.6 查询联系人	26
4.4.7 统计联系人数量	27
4.4.8 退出与最小化	27
参考文献	28

1 引言

数据结构是计算机科学中的基础课程，它研究了在计算机内存中组织和管理数据的方法和技术。数据结构不仅仅是一种存储和访问数据的方式，它还提供了解决各种实际问题的有效算法和操作。

正确的数据结构选择可以提高算法的效率。在程序设计的过程中，选择适当的数据结构是一项重要工作。许多大型系统的编写经验显示，程序设计的困难程度与最终成果的质量与表现，取决于是否选择了最适合的数据结构。

通讯录是一种用于管理个人或组织联系人信息的工具，它允许用户存储、查找和更新各种联系人的详细信息，如姓名、电话号码、电子邮件地址等。通讯录的功能广泛应用于个人和商业场景，包括个人手机、电子邮件客户端、社交媒体平台等。基于双链表数据结构的双链表通讯录管理系统为实现通讯录电子化提供了一种有效的解决方案。双链表是一种动态数据结构，它由节点组成，每个节点都包含一个联系人的基本信息、一个前指针和一个后指针。这种数据结构能够快速插入、删除和访问节点，非常适合通讯录的管理。

本次课程设计通过设计一个基于双向循环链表的双链表通讯录，实现了添加联系人、插入联系人、删除联系人、查询联系人和统计联系人数量等功能。

本报告分为四部分，分别介绍了课程设计背景，课程设计详细分析，课程设计程序详细设计和系统设计四部分。

2 课程设计分析

2.1 问题分析

2.1.1 以双链表的数据结构存储

题目要求用双向循环链表来存储通讯录信息。每个节点应该包含姓名，手机号，QQ 号和邮箱等信息。（为每个联系人增加备注信息，但不强制要求为 nullptr。）

2.1.2 实现通讯录相关的操作

设计相关的函数，实现通讯录信息的添加、插入、删除、查询和统计等功能，应用数据结构知识优化上述操作，提高程序效率。

2.1.3 对输入的数据进行校验

对输入的数据进行校验，需要对通讯录常用的手机号、邮箱进行校验，完善课程设计要求，提高通讯录信息的准确率。

2.1.4 设计友好的用户交互界面

通过设计一个包含相关操作的菜单来提示用户输入并起到完善程序功能的作用。

2.2 需求分析

2.2.1 存储结构分析

程序需要使用双向循环链表作为存储结构，它可以在常数时间内在任意位置插入和删除节点。用户可能会频繁地添加、插入和删除联系人信息，使用双链表可以使程序效率更高。

通讯录需要存储联系人的基本信息包括姓名，手机号，QQ 号和邮箱等信息，用于实现查询、统计等功能。

2.2.2 功能分析

程序需要实现通讯录信息的添加、插入、删除、查询和统计等功能。

添加功能应该能够在通讯录中添加新的联系人信息。插入功能应能在通讯录中插入一个完整的联系人基本信息。删除功能需要能够删除指定联系人的所有信息。查询功能应该能够根据指定条件查找联系人信息，用户应该能够输入查询条件（如姓名、手机号等），并根据这些条件查找符合条件的联系人信息。统计功能需要实现统计通讯录中的联系人数量的功能。

2.2.3 用户界面设计

设计合理的用户操作界面，提示用户进行功能选择。并且在合适的位置进行提示；设置相关的数据验证功能，针对易出错的手机号和邮箱进行数据校验，在输入错误结果时给出相应的错误提示，帮助用户纠正错误。

2.3 算法设计

2.3.1 双链表数据结构

定义双链表节点，建立通讯录节点结构。结构需包含人员通讯录包括姓名，手机号，QQ 号和邮箱等基本信息，以及双链表节点的前指针和后指针。

2.3.2 功能实现算法

添加联系人：在链表中创建一个新节点，并将其插入到链表的末尾。

插入联系人：在指定位置插入一个新节点，将原位置及其后面的节点向后移动。

删除联系人：根据联系人姓名，在链表中查找并删除对应的节点。

查询联系人：根据联系人姓名、手机号、QQ 号或邮箱，遍历链表查找对应的节点，并输出联系人。

统计联系人数量：遍历链表，计算链表中节点的数量。

3 程序设计

3.1 程序框架

首先定义结构体的相关成员，包括联系人的基本信息例如联系人的姓名、手机号等，双链表的前指针和后指针。

定义通讯录所需要的功能函数，添加联系人，删除联系人，插入联系人，查询联系人以及对手机号和邮箱的数据验证。

程序流程如图 3.1 所示。

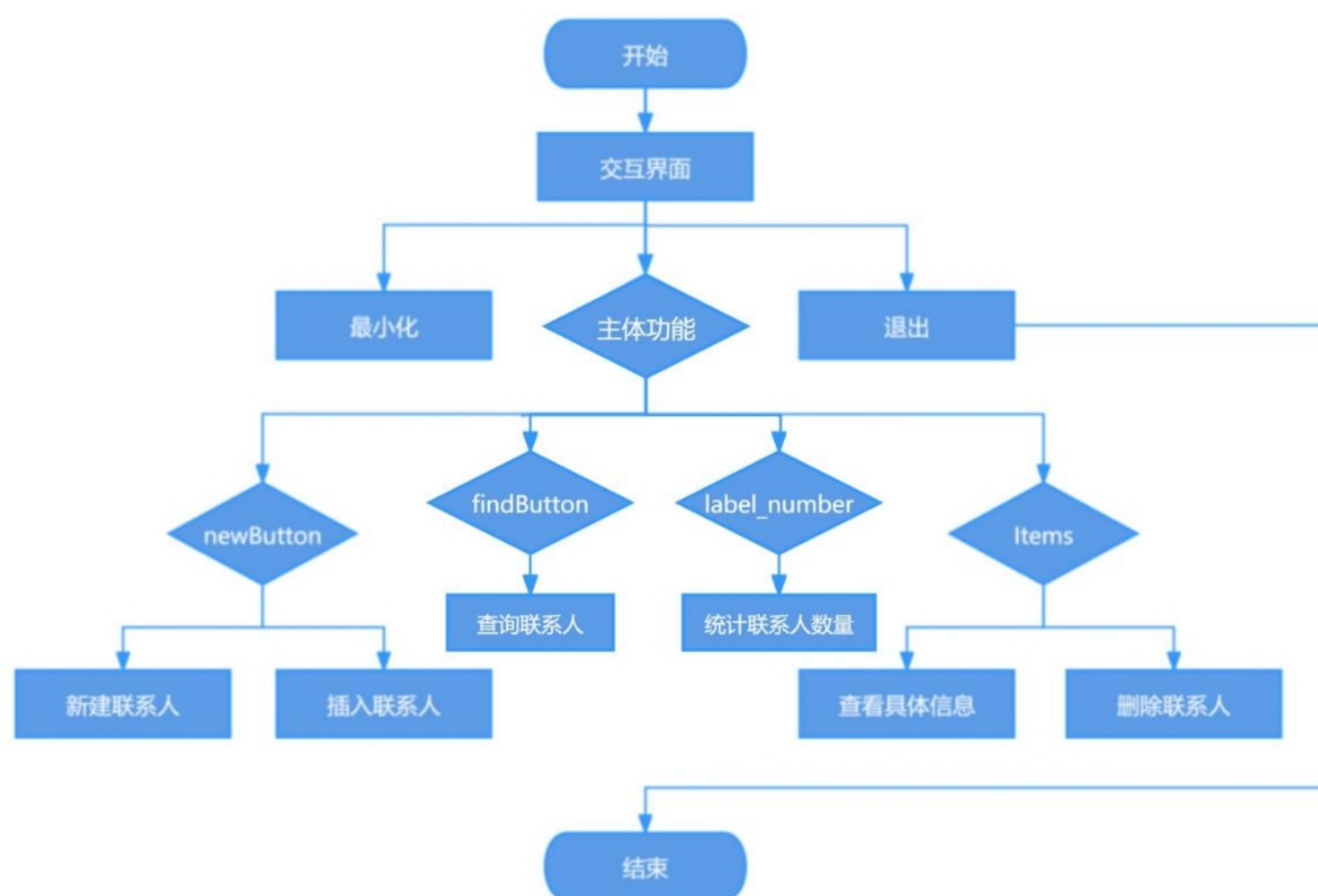


图 3.1 程序流程图

3.2 模块设计

3.2.1 链表节点结构体定义

该节点需要包含联系人的姓名、手机号、QQ 号、邮箱、备注（备注默认为“无”）这些基本信息，同时还需要包含双链表数据结构的前指针和后指针。

3.2.2 数据验证

使用正则表达式模式 `^\\d{11}$` 或 `\\d{3,4}-?\\d{7,8}` 对电话号码进行验证。第一种模式表示电话号码必须由 11 位数字组成，以确保满足手机号的基本格式要求。第二种模式表示电话号码可以是 3 到 4 位数字的区号，后面可选地跟一个短横线，再接 7 到 8 位数字的本地号码。函数通过调用 `exactMatch` 方法将传入的 `phone` 参数与这两个正则表达式模式进行匹配，满足任意一个即可通过验证。

使用正则表达式模式 `\\b[A-Z0-9._%+-]+@[A-Z0-9.-]+\\.\\b[A-Z]{2,4}\\b` 对邮箱进行验证。该模式表示邮箱必须包含一个或多个字母、数字、点号、百分号、加号、减号或下划线，后面紧跟一个 `@` 符号，再接一个或多个字母、数字、点号或连字符，最后是一个点号和 2 到 4 位的大写字母后缀。该正则表达式设置为不区分大小写。函数通过调用 `exactMatch` 方法将传入的 `email` 参数与正则表达式模式进行匹配。数据验证流程如图 3.2 所示。

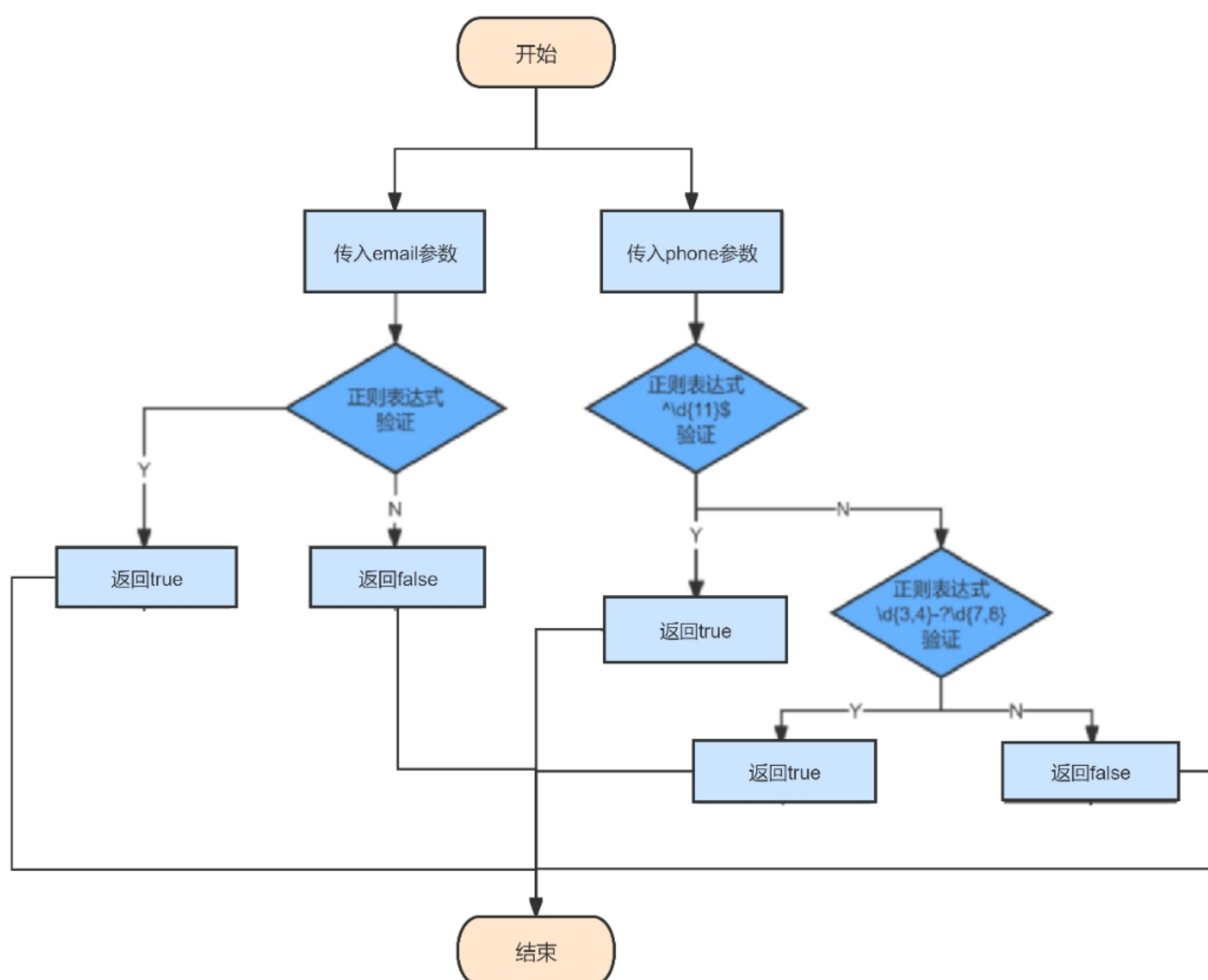


图 3.2 数据验证流程图

3.2.3 新建联系人

使用 `is_valid_phone` 函数验证传入的电话号码 `phone` 是否符合格式要求。如果电话号码格式无效，发出错误信号 `error("电话号码格式无效。")` 并返回。

使用 `is_valid_email` 函数验证传入的邮箱 `email` 是否符合格式要求。如果邮箱格式无效，发出错误信号 `error("邮箱格式无效。")` 并返回。

使用 `find_ability` 函数根据姓名 `name` 查找联系人。如果联系人已存在，发出错误信号 `error("联系人已存在。")` 并返回。

创建一个 `Ability` 对象 `new_ability`，并用传入的参数初始化该对象。然后创建一个 `Node` 对象 `new_node`，并将 `new_ability` 赋值给该节点。

检查头节点 `head` 是否为空。如果通讯录为空，即 `head` 为 `nullptr`，表示这是第一个联系人节点。将新节点设为头节点 `head` 和尾节点 `tail`，并设置新节点的前驱节点 `prev` 和后继节点 `next` 都指向自身，形成双向循环链表。

如果通讯录不为空，即已存在其他联系人，将新节点插入到尾节点 `tail` 和头节点 `head` 之间：

- (1) 将新节点的前驱节点 `prev` 指向当前的尾节点 `tail`，后继节点 `next` 指向头节点 `head`。
- (2) 将当前尾节点 `tail` 的后继节点 `next` 指向新节点 `new_node`，将头节点 `head` 的前驱节点 `prev` 指向新节点 `new_node`。
- (3) 将尾节点 `tail` 更新为新节点 `new_node`。

如果参数 `emit_signal` 为 `true`，发出联系人添加信号 `ability_added()`。以便于后续在界面文件中实现快速调用。

新建联系人流程如图 3.3 所示。

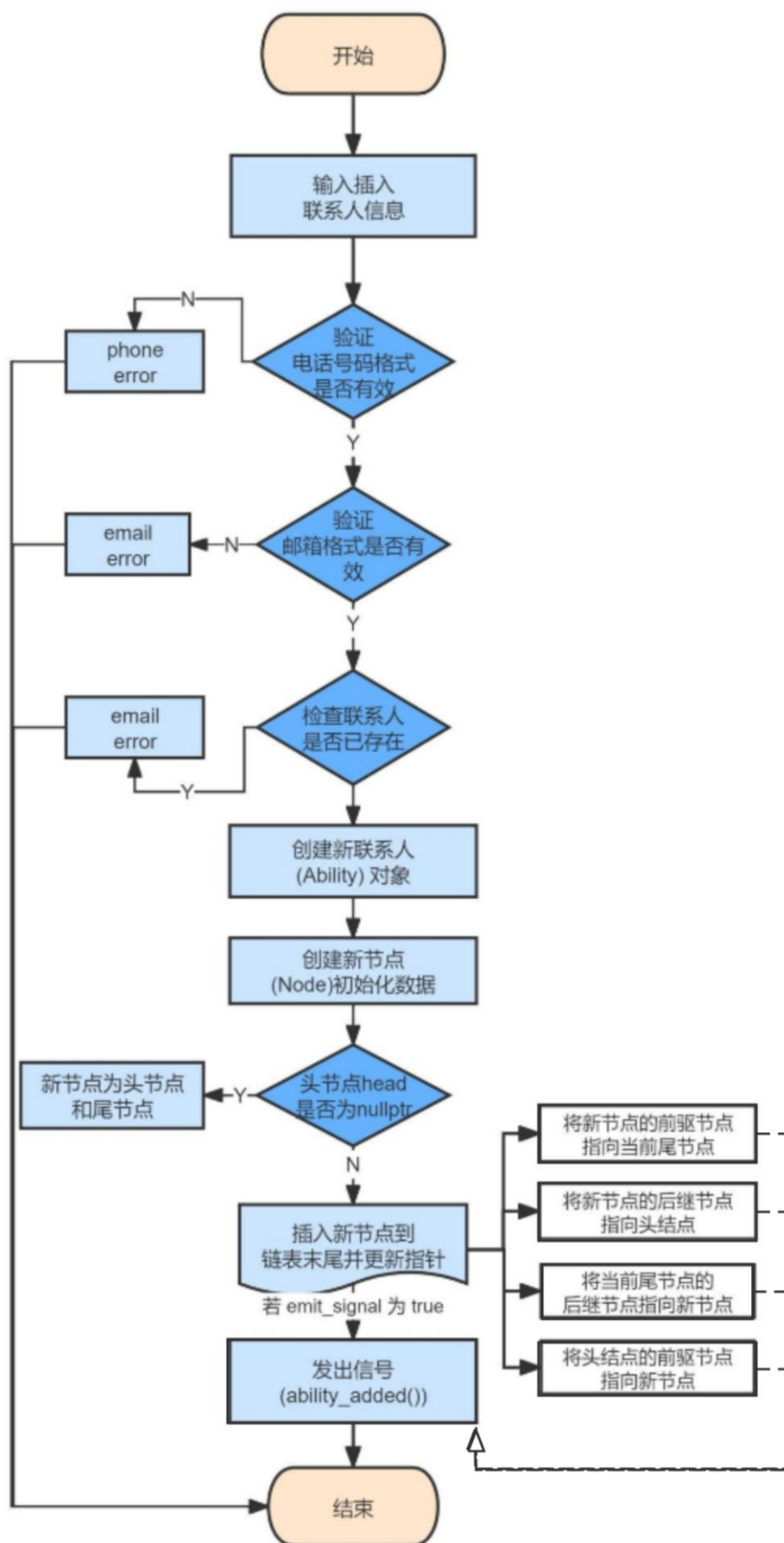


图 3.3 新建联系人流程图

3.2.4 插入联系人

使用 `is_valid_phone` 函数验证传入的电话号码 `phone` 是否符合格式要求。如果电话号码格式无效，发出错误信号 `error("电话号码格式无效。")` 并返回。

使用 `is_valid_email` 函数验证传入的邮箱 `email` 是否符合格式要求。如果邮箱格式无效，发出错误信号 `error("邮箱格式无效。")` 并返回。

使用 `find_ability` 函数根据姓名 `name` 查找联系人。如果联系人已存在，发出错误信号 `error("联系人已存在。")` 并返回。

创建一个 `Ability` 对象 `new_ability`，并用传入的参数初始化该对象。然后创建一个 `Node` 对象 `new_node`，并将 `new_ability` 赋值给该节点。检查头节点 `head` 是否为空。如果链表为空，将新节点设为头节点 `head` 和尾节点 `tail`，并设置新节点的前驱节点 `prev` 和后继节点 `next` 都指向自身，形成双向循环链表。

确保传入的插入位置 `position` 在有效范围内。如果位置小于 0，将位置设为 0；如果位置大于链表大小 `size`，将位置设为链表大小 `size`。创建一个指针 `current` 并将其指向头节点 `head`。使用循环将 `current` 指针移动到指定插入位置 `position`。

将新节点的前驱节点 `prev` 指向当前节点的前驱节点，将新节点的后继节点 `next` 指向当前节点。将当前节点的前驱节点的后继节点 `next` 指向新节点，将当前节点的前驱节点 `prev` 指向新节点。

如果插入位置为 0，更新头节点 `head` 为新节点 `new_node`。如果插入位置为链表大小 `size`，更新尾节点 `tail` 为新节点 `new_node`。将链表大小 `size` 自增 1。

如果参数 `emit_signal` 为 `true`，发出联系人添加信号 `ability_added()`。以便于后续在界面文件中实现快速调用。

插入联系人流程如图 3.4 所示。

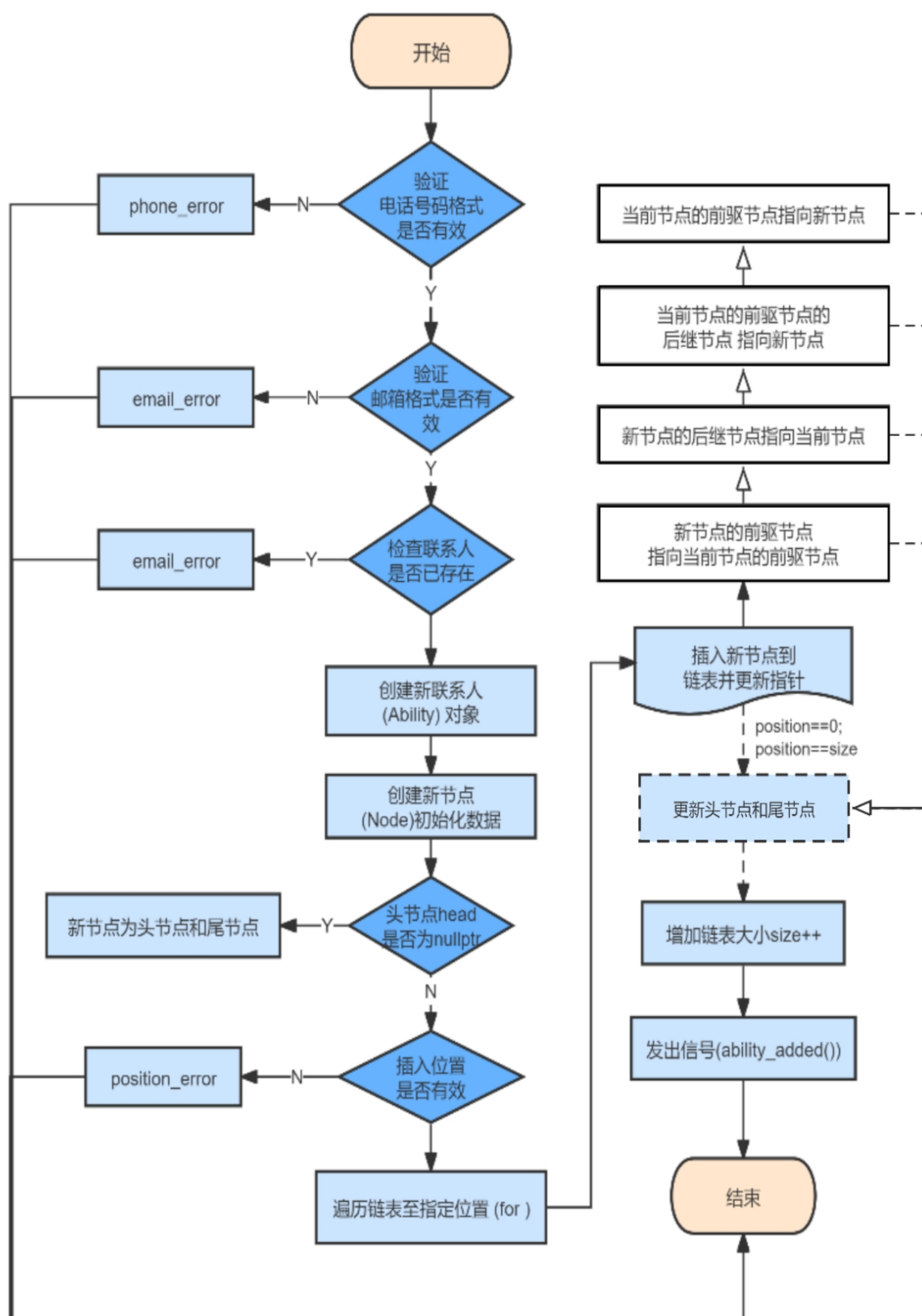


图 3.4 插入联系人流程图

3.2.5 删除联系人

首先检查通讯录是否为空，即头节点 head 是否为 nullptr。如果通讯录为空，直接返回，表示没有联系人可以删除。

创建一个指针 current 并将其指向头节点，用于遍历通讯录链表。使用 do-while 循环遍历链表，至少执行一次循环。在循环中，判断当前节点的姓名是否与要删除的姓名匹配。

如果找到匹配的联系人节点：

1. 删除唯一节点：如果链表中只有一个节点（即 current 同时是 head 和 tail），将 head 和 tail 设为 nullptr。

2. 删除非唯一节点：调整前驱节点和后继节点的指针，移除当前节点：

- (1) 将当前节点的前驱节点的 next 指针指向当前节点的后继节点。
- (2) 将当前节点的后继节点的 prev 指针指向当前节点的前驱节点。
- (3) 如果当前节点是 head，更新 head 指向当前节点的后继节点。
- (4) 如果当前节点是 tail，更新 tail 指向当前节点的前驱节点。

删除节点：删除 current 节点并释放其内存空间，发出删除信号 ability_deleted()，并返回。由此，以便于后续在界面文件中实现快速调用。

如果当前节点不匹配，继续移动到下一个节点，直到遍历完整个链表。当遍历到头节点时结束循环，表示已遍历完整个通讯录链表。

如果循环结束后仍未找到匹配的联系人姓名，表示未找到该联系人。

以便于后续在界面文件中实现快速调用。

删除联系人流程如图 3.5 所示。

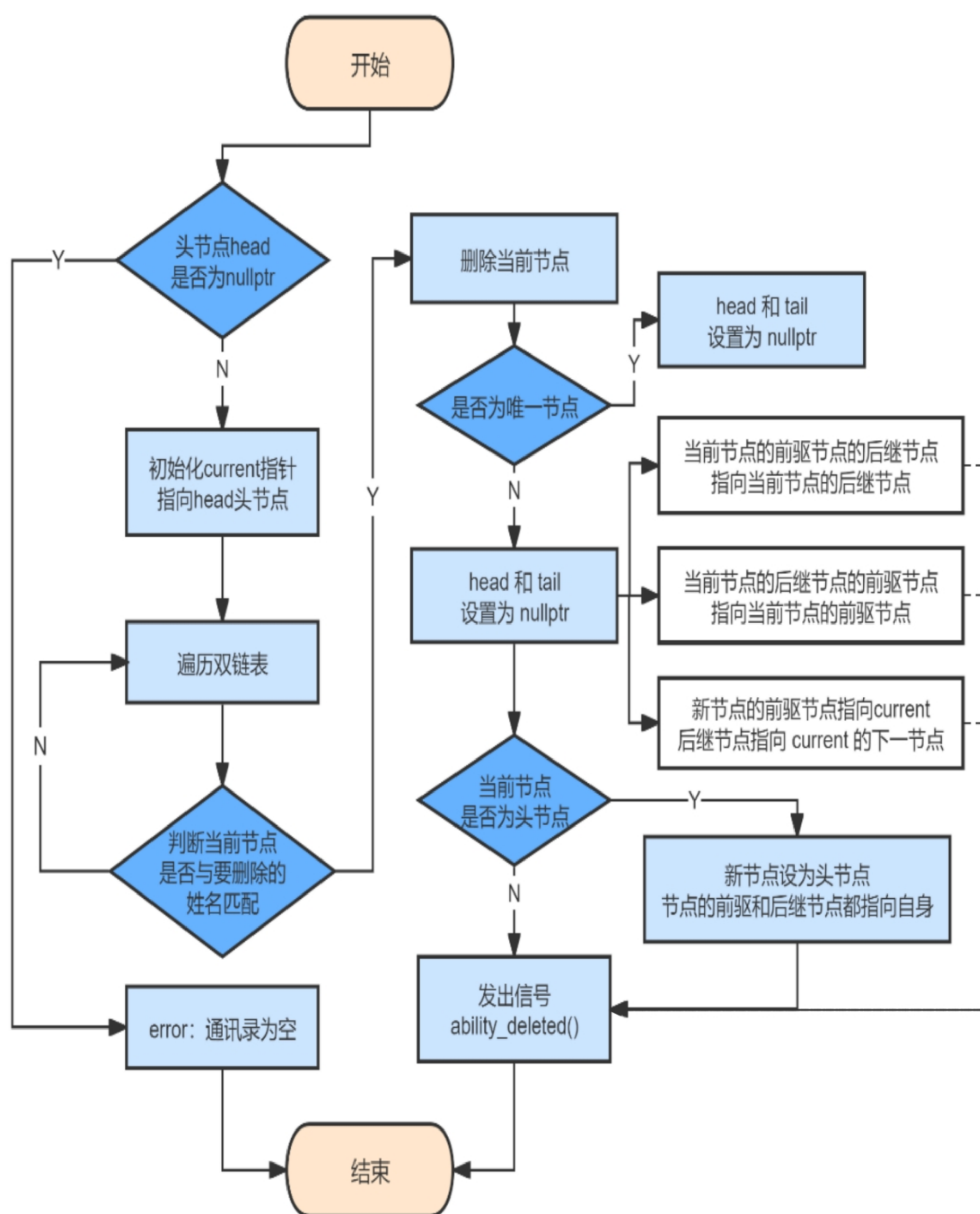


图 3.5 删除联系人流程图

3.2.6 查询联系人

首先检查通讯录是否为空，即头节点 head 是否为 nullptr。如果通讯录为空，返回一个默认的 Ability 对象，表示未找到匹配的联系人。

创建一个指针 current 并将其指向头节点 head，用于遍历通讯录链表。这样可以确保从链表的开头开始进行遍历操作。

使用 do-while 循环遍历链表，保证至少执行一次循环。该循环在结束之前会访问链表中的每一个节点。在循环中，依次判断当前节点的 name、phone、qq 和 email 是否与搜索查询关键字 key 匹配。如果找到匹配项，则表示找到了相应的联系人。

如果找到匹配的联系人，返回 current->ability 对象，表示已找到匹配的联系人。程序将退出循环并返回该联系人信息；如果未找到匹配的联系人，移动指针 current 指向下一个节点，即 current->next，并继续进行下一次循环。当 current 再次指向头节点 head 时，结束循环。此时表示已经遍历完整个链表而未找到匹配的联系人。

如果循环结束后仍未找到匹配的联系人，返回一个默认的 Ability 对象，表示未找到匹配的联系人。

查询联系人流程如图 3.6 所示。

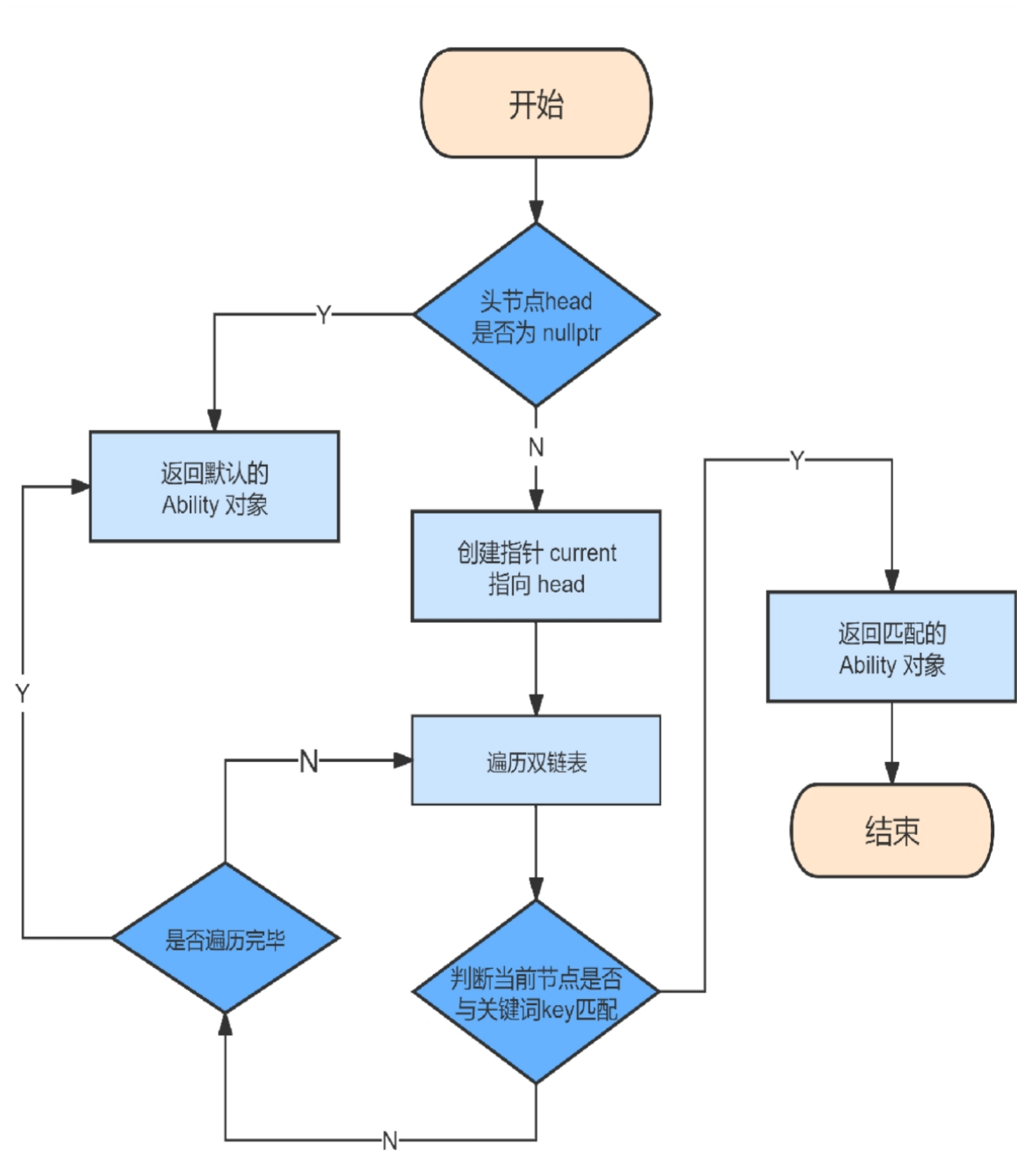


图 3.6 查询联系人流程图

3.2.7 统计联系人

首先初始化计数变量 `count` 为 0，用于记录通讯录中联系人的数量。

创建一个指针 `current` 并将其指向头节点 `head`，用于遍历通讯录链表。检查头节点 `head` 是否为空。如果 `head` 为 `nullptr`，表示通讯录中没有联系人，直接返回计数变量 `count`。

使用 `do-while` 循环遍历链表，保证至少执行一次循环。在循环中进行以下操作：

- (1) 自增计数变量：将计数变量 `count` 自增 1，表示找到一个联系人。
- (2) 移动指针：将指针 `current` 移动到下一个节点，即 `current->next`。

当 `current` 再次指向头节点 `head` 时，结束循环，表示已遍历完整个通讯录链表。返回计数变量 `count`，表示通讯录中联系人的数量。

统计联系人流程如图 3.7 所示。

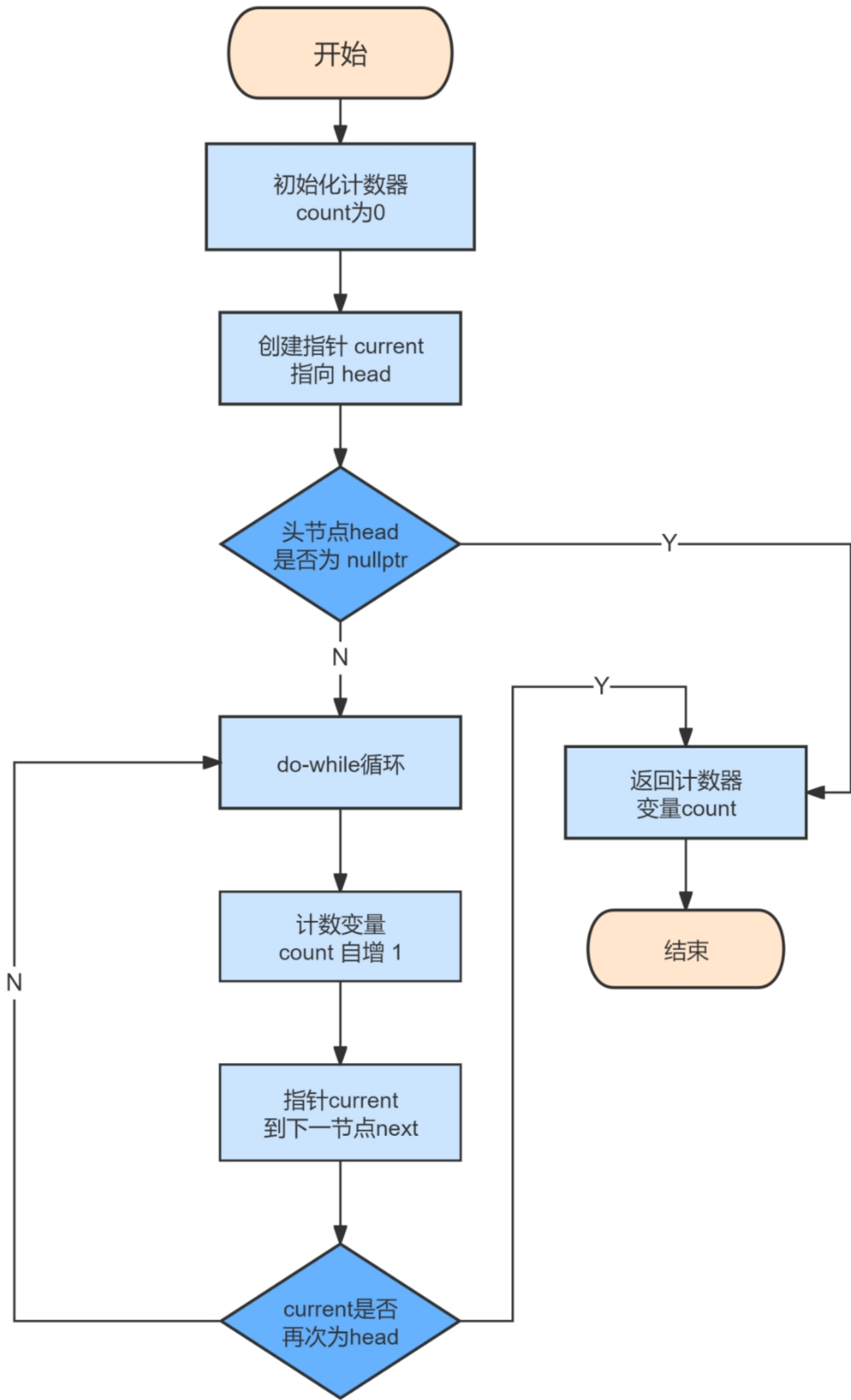


图 3.7 统计联系人流程图

4 系统测试

4.1 测试环境

编译工具: CMake

编译软件: Qt Creator

4.2 测试数据

表 4.1 数据验证功能测试变量

姓名	手机号	QQ 号	邮箱	备注	位置
陈祥欣	15541609703	1939411884	1939411884@com		
陈祥欣	15541609703	1939411884	1939411884@qq	我	
陈祥欣	15541609703	1939411884	1939411884@		
妈妈	1384062	1837487400	1837487400@qq.com	mother	
妈妈	13840627185	1837487400	1837487400@qq.com		2.7
妈妈	13840627185	1837487400	1837487400@qq.com		5
郑羽婷	1394135599555	321889544	321889544@qq.com	09	
郑羽婷	13941355995	321889544	321889544@qqcom	09	

表 4.2 最终存放本地数据库测试变量

姓名	手机号	QQ 号	邮箱	备注
test	123-45678910	12345	12345@qq.com	
陈祥欣	15541609703	1939411884	1939411884@qq.com	我
妈妈	13840627185	1837487400	1837487400@qq.com	mother
郑羽婷	13941355995	321889544	321889544@qq.com	09

注: 除“备注”外, 姓名、手机号、QQ 号、邮箱四项默认均为 nullptr, 是空字

字符串，没有默认值，必须填写。

4.3 本地数据库

resung	2024/6/29 22:15	文件夹
untitled_autogen	2024/6/29 22:15	文件夹
abilities.txt	2024/6/29 22:02	文本文档
cmake_install.cmake	2024/6/25 21:02	CMake 源文件
CMakeCache.txt	2024/6/25 21:02	文本文档

图 4.2 本地数据库文件格式

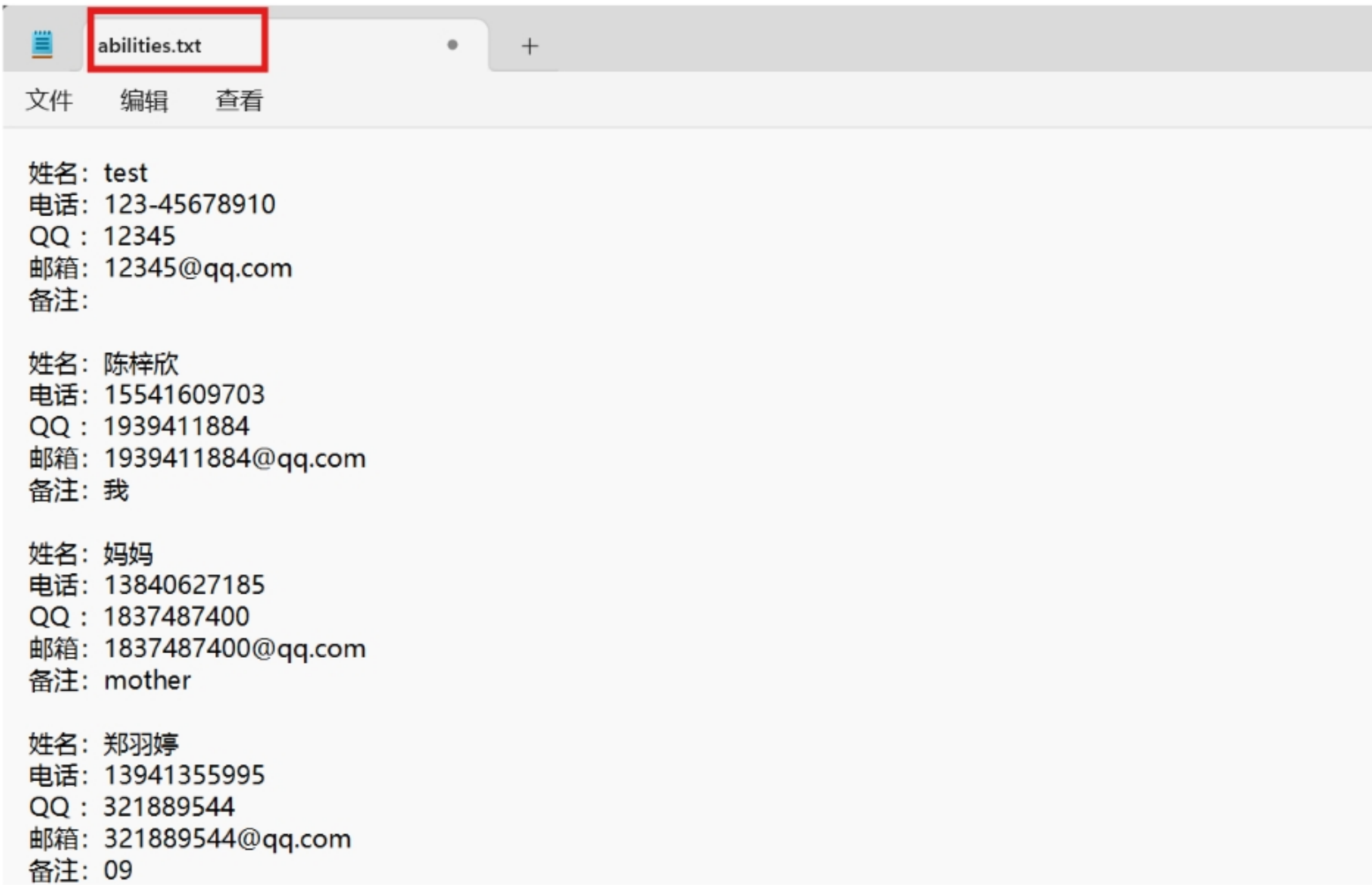


图 4.3 联系人信息本地储存效果

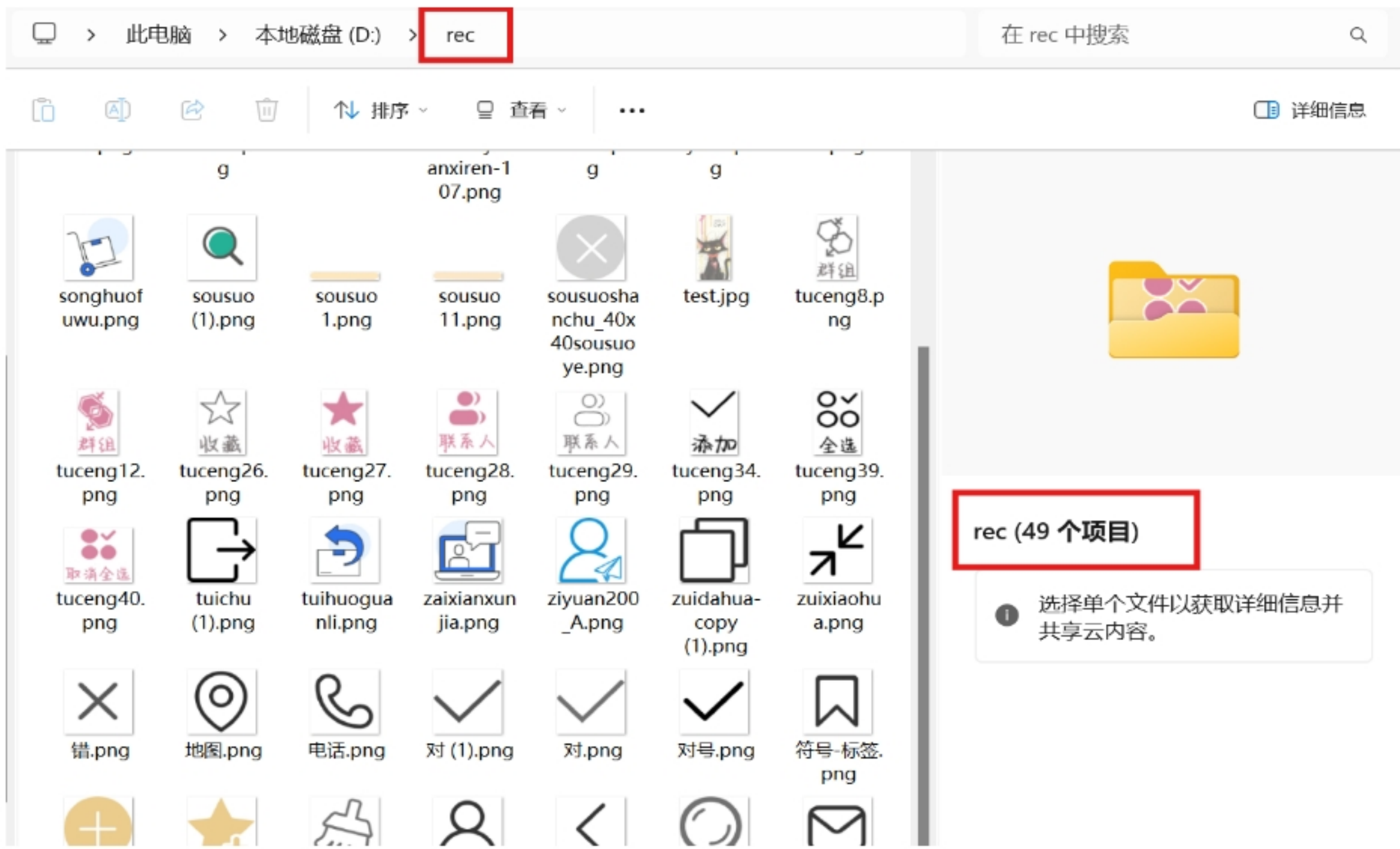


图 4.4 项目所需界面资源本地链接

4.4 测试过程演示

4.4.1 通讯录管理系统页面展示：

(1) 主页面与联系人页面



图 4.5 通讯录管理系统页面展示 (1)

(2) 群组页面与收藏页面



图 4.5 通讯录管理系统页面展示 (2)

4.4.2 新建联系人



图 4.6 添加联系人演示

4.4.3 错误信号验证

(1) 邮箱格式无效

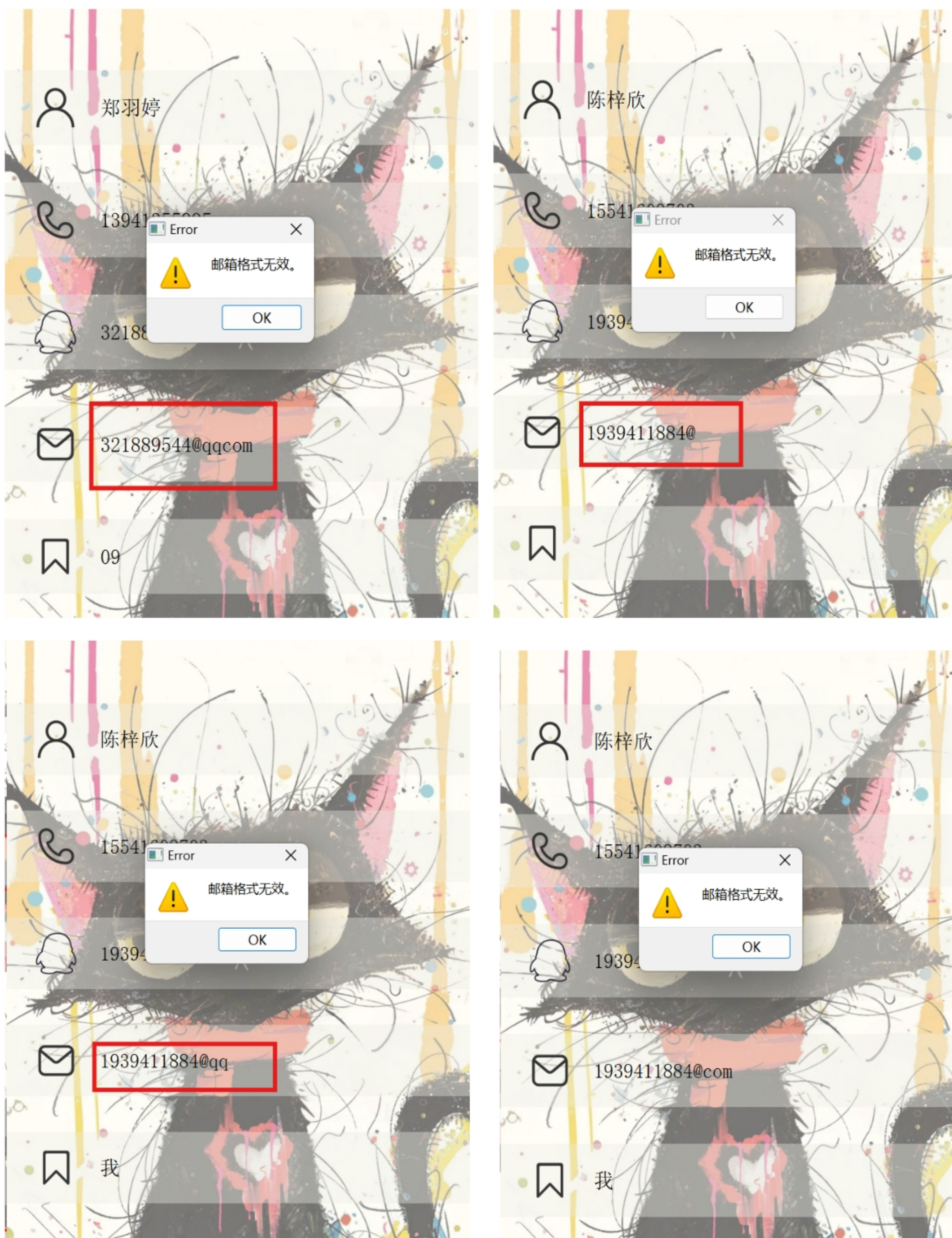


图 4.7 错误信号验证演示 (1)

(2) 电话号码格式无效

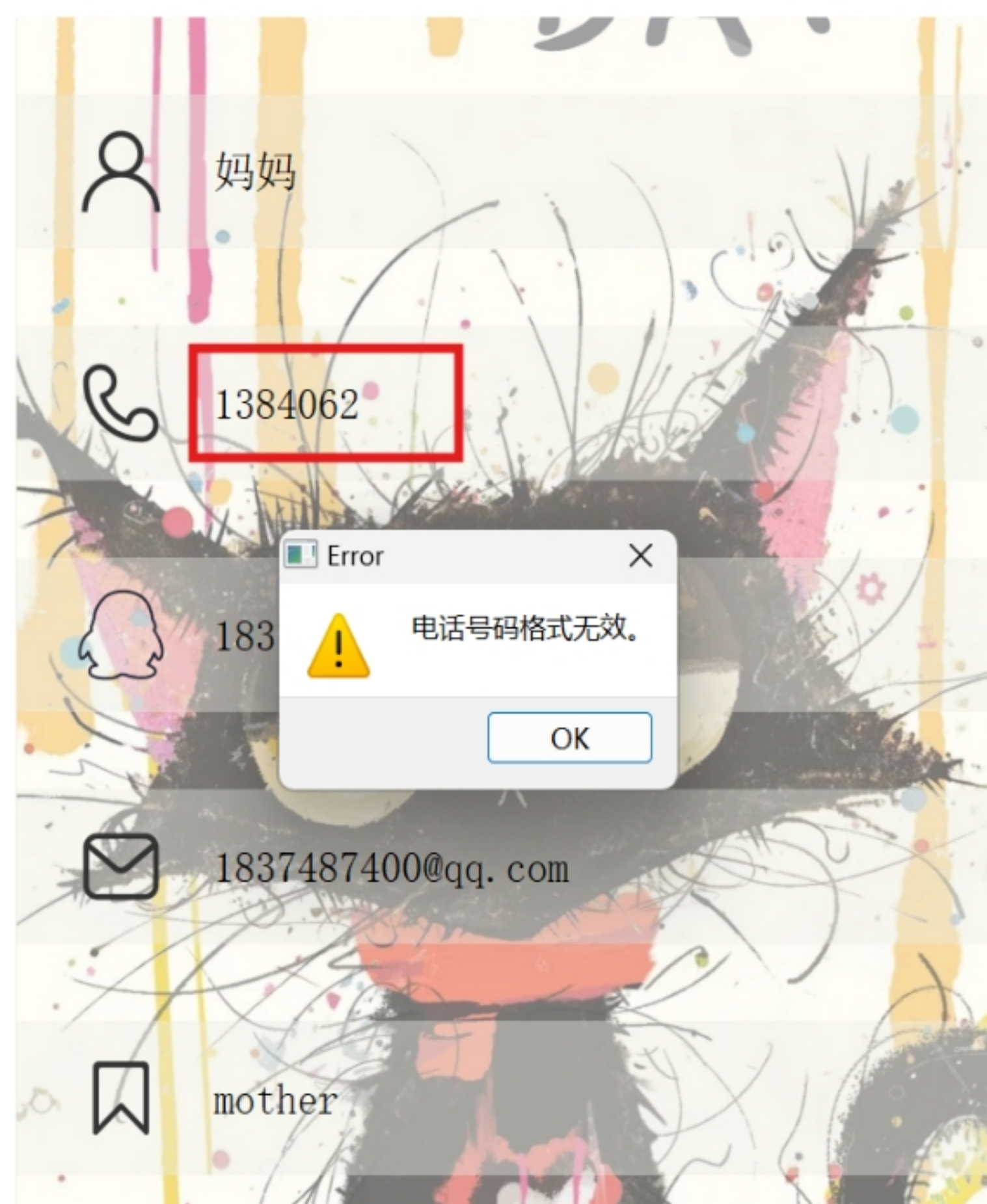
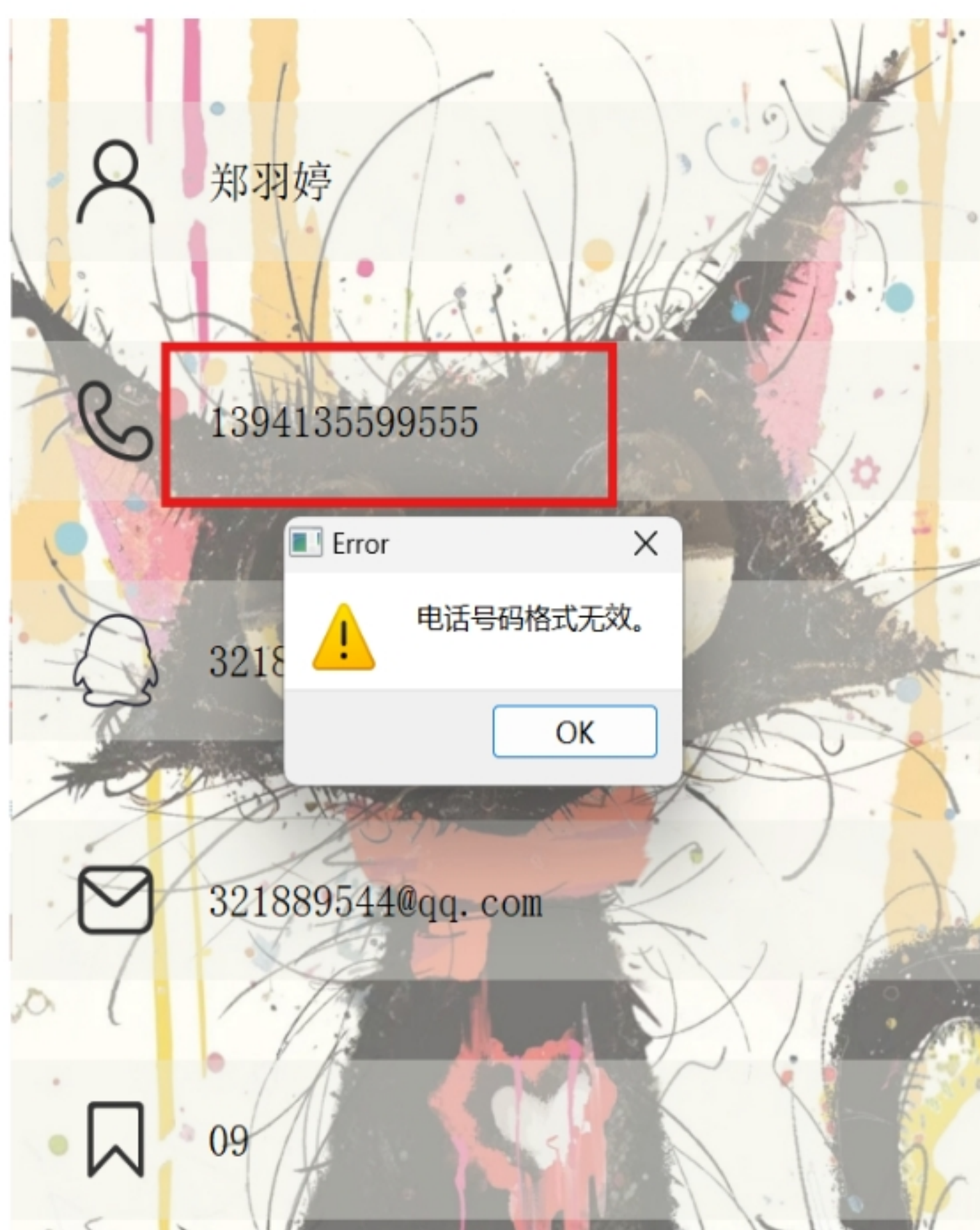


图 4.8 错误信号验证演示 (2)

(3) 联系人已存在与插入节点无效

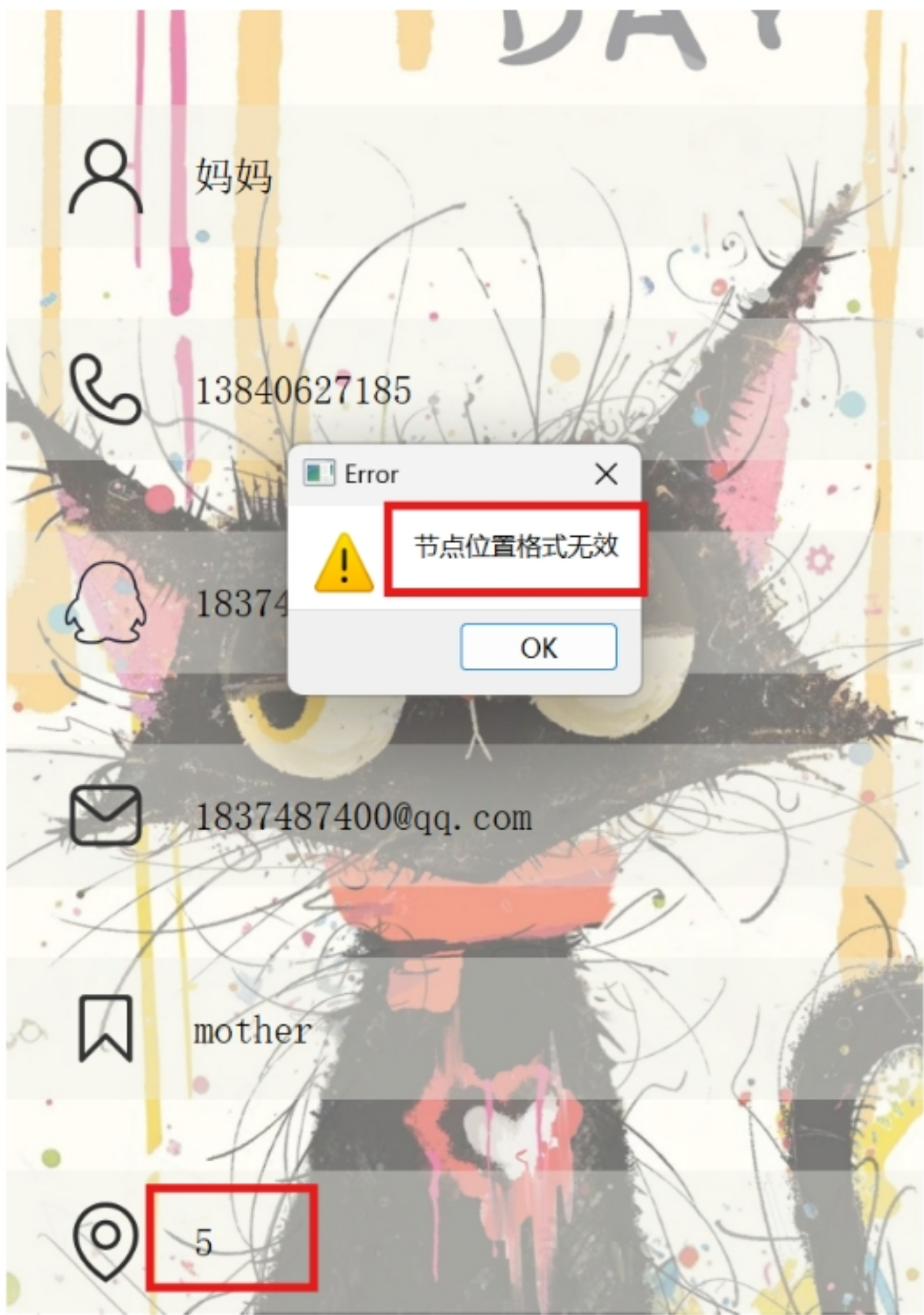
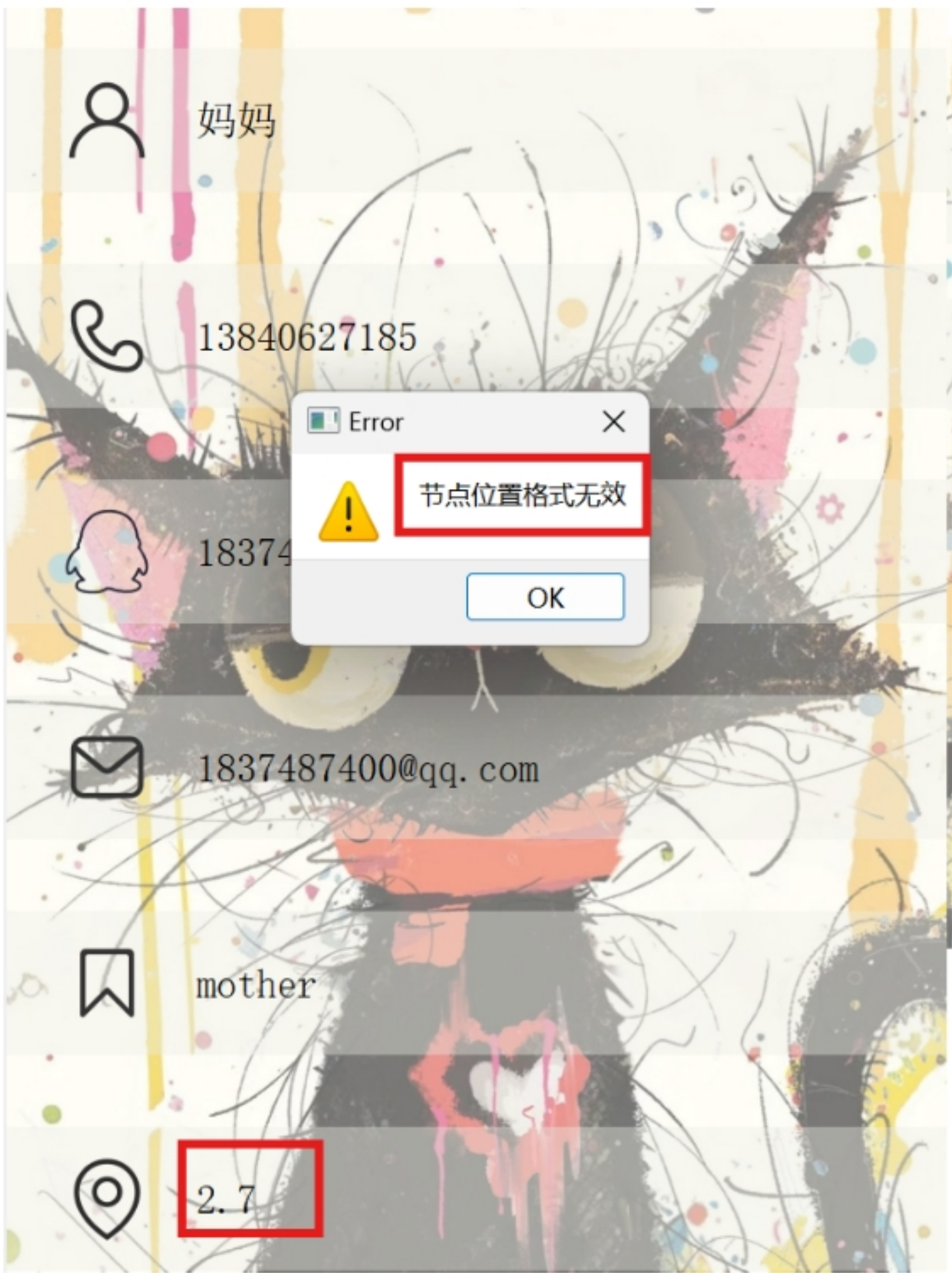


图 4.9 错误信号验证演示 (2)

4.4.4 插入联系人



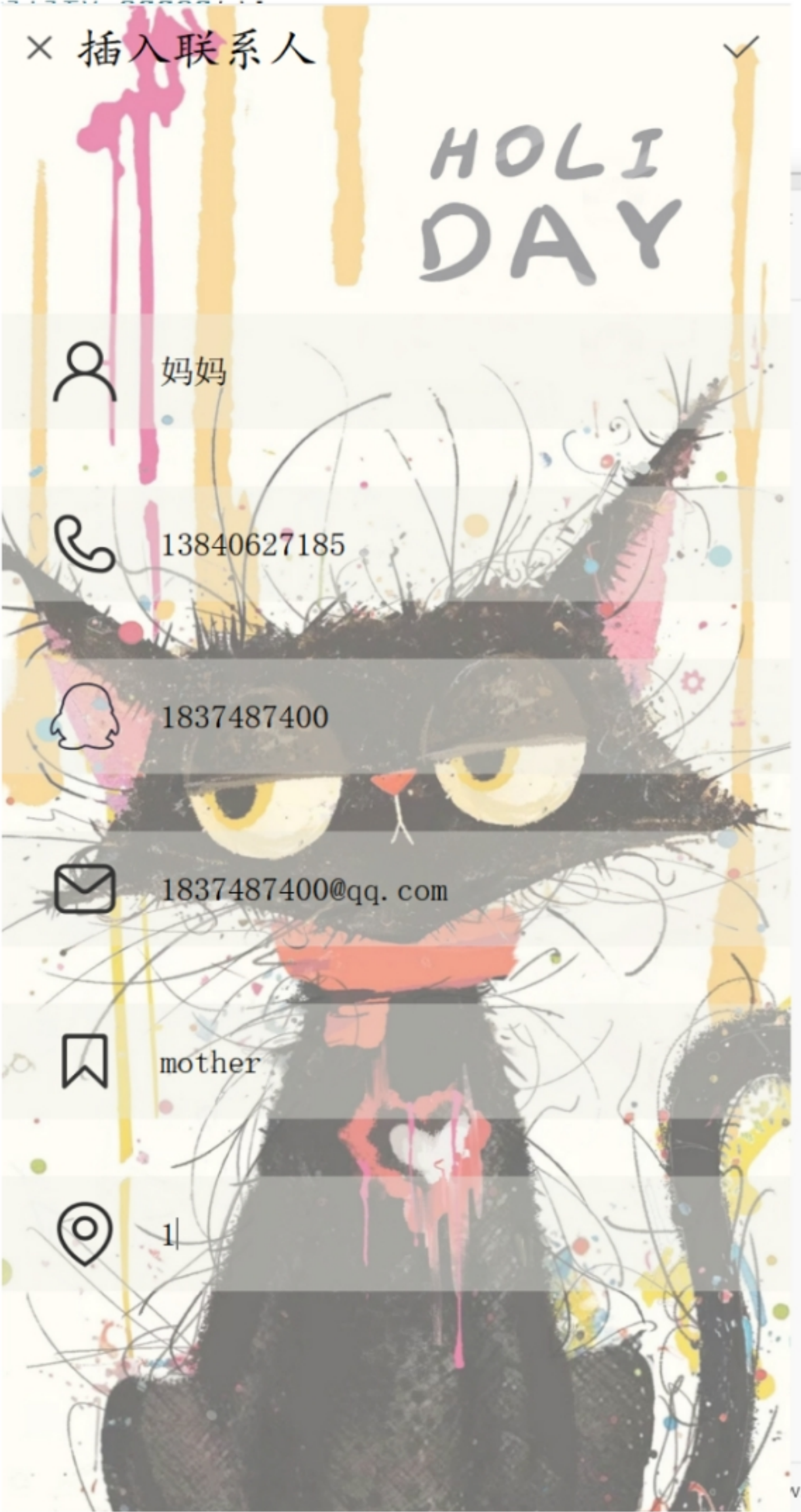


图 4.10 插入联系人演示

4.4.5 删除联系人



图 4.11 删除联系人演示

4.4.6 查询联系人



图 4.12 查询联系人演示

4.4.7 统计联系人数量



图 4.13 统计联系人数量

4.4.8 退出与最小化



图 4.14 退出



图 4.15 最小化

参考文献

- [1]陈卫卫、王庆瑞, 数据结构与算法(第2版), 北京: 高等教育出版社。
- [2] 严蔚敏等, 数据结构(C语言版), 北京: 清华大学出版社, 1997。
- [3] 王晓东著, 算法设计与分析(第4版), 北京: 清华大学出版社。
- [4]李卓, 计算机算法设计及数据结构离散性研究[J], 科技资讯, 2024, 22(05): 51-53. DOI: 10.16661/j.cnki.1672-3791.2311-5042-1903.
- [5]沈华, 数据结构、算法和程序之间关系的探讨[J], 计算机教育, 2013(04): 58-61. DOI: 10.16512/j.cnki.jsjyy.2013.04.026.
- [6]杨东, 链表在程序设计中的使用方法与技巧[J], 电脑编程技巧与维护, 2012(19): 21-24. DOI: 10.16184/j.cnki.comprg.2012.19.006.
- [7]熊杰, 刘新德, 冯仁剑, 嵌入式图形用户界面系统的设计与实现[J], 计算机工程与设计, 2012, 33(07): 2602-2606. DOI: 10.16208/j.issn1000-7024.2012.07.017.
- [8]吴绍兵, 计算思维和程序设计能力的培养[J], 计算机教育, 2011(16): 11-14+25. DOI: 10.16512/j.cnki.jsjyy.2011.16.019.
- [9]张明伟, 双向循环有序链表的设计与实现[J], 中国科技信息, 2009(16): 89.